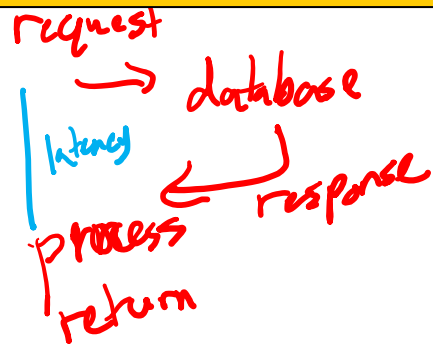


Web Server

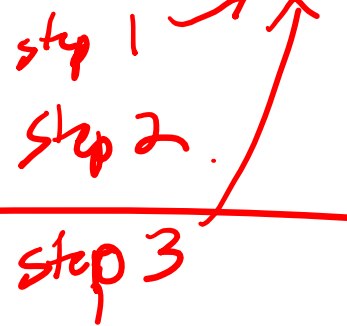
Not always directly about parallelism! Can improve throughput by doing useful work during latency while waiting for external resources.



Thread

Drawback: Additional complexity. Example: Can multiple concurrent tasks access, modify, and depend on the same variable?!

Function



Lecture 16

Intro to Concurrency

Concurrency means multiple “tasks” whose timelines overlap with each other. Generally speaking, the benefit is improved throughput and/or reduced latency.

Example Code and Exercises

<https://github.com/eecs498-software-design/lecture>

Concurrency vs. Parallelism

Concurrency

Parallel

is

task 1

pause



task 2



task 3



Concurrent, but not parallel. The tasks are interleaved, but never actively executing at the same time.

Concurrent and parallel. There are some places where multiple tasks are executing at the same time.



Single-Threaded vs. Multithreaded

TS/JS
single-threaded

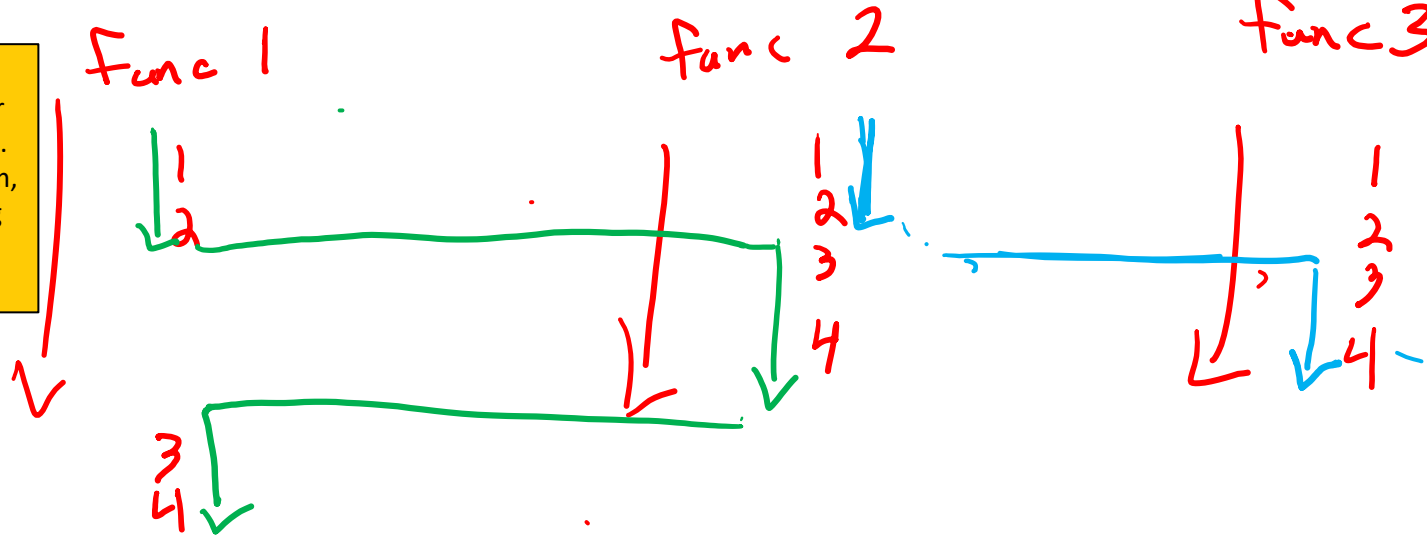
Thread : execution resource.

single threaded

multi-threaded

One execution resource used for overlapping tasks. No true parallelism, though switching rapidly may give that appearance.

Additional execution resources. This can improve performance IF work is parallelizable.

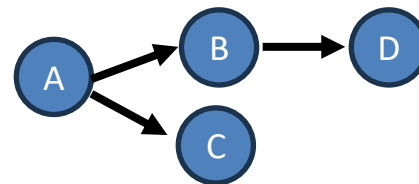


Synchronous vs. Asynchronous

- Synchronicity is most meaningful when considering specific “things that happen”, e.g. A, B, and C.
- If the ordering is definitely $A \rightarrow B \rightarrow C$, they’re synchronous.
- We could also have e.g. $A \rightarrow B$ and $A \rightarrow C$ and $B \rightarrow D$.
 - Some are synchronous some are asynchronous.
- Contrast to blocking vs. non-blocking.

Synchronous vs. Asynchronous

- Consider specific “things that happen”, e.g. A, B, and C.
- If the ordering is definitely $A \rightarrow B \rightarrow C$: they’re **synchronous**.
- We could also have e.g. $A \rightarrow B$ and $A \rightarrow C$ and $B \rightarrow D$:
 - **Synchronous**: A/B, A/C, and A/D (by transitivity)
 - **Asynchronous**: B/C, B/D, C/D
- There are several high-level patterns of synchronization, e.g.:
 - Sequencing: Don’t start (or pause) task X until one or more other tasks has finished.
 - Mutual Exclusion: Ensure $A_1, A_2, \dots, A_n$ and $B_1, B_2, \dots, B_n$ don’t interleave.
 - Barrier: If A, B, C have multiple parts, ensure all finish each part before any move on.
- Sometimes these arise naturally from control-flow.
Otherwise, use synchronization constructs to enforce them.



NOTE: We did not cover a categorization of synchronization patterns extensively in W26. They won’t be on the final exam.

Blocking vs. Non-Blocking

- When a program performs I/O or some other “external” operation, how does **control flow** respond? And, which control flow are you talking about?

Blocking: Pause at that line until the result is ready.

```
import { readFileSync } from 'fs';
console.log('Welcome to line counter!');
const str = readFileSync('input.txt', 'utf-8');
const lines = str.split('\n');
console.log(`File has ${lines.length} lines.`);

console.log("Also, let's print some numbers!");
for (let i = 0; i < 10; i++) {
  console.log(i);
}
```

Non-Blocking: Keep going, come back later for the result.

```
import { readFile } from 'fs';
console.log('Welcome to line counter!');
readFile('input.txt', 'utf-8', (err, data) => {
  const lines = data.split('\n');
  console.log(`File has ${lines.length} lines.`);
});

console.log("Also, let's print some numbers!");
for (let i = 0; i < 10; i++) {
  console.log(i);
}
```

Terminology

→ What if examples below are in a function call, followed by D? (not awaited)

The last couple slides are *my* preferred framing. Nevertheless you'll encounter:

- Taxonomies of the 4 combinations of sync/async and blocking/non-blocking.
- Loose terminology like “this blocks for synchronization” or “call this function asynchronously”.

Do your best to infer from context: What is being blocked? Which A and B are sync or async?

①

```
import { readFileSync } from 'fs';
console.log('Welcome to line counter!');

A const str = readFileSync('input.txt');
B const lines = str.split('\n');
  console.log(`${lines.length} lines.`);

C console.log("Also print some numbers!");
  for (let i = 0; i < 10; i++) {
    console.log(i);
  }
```

blocks everything

sync: A → B → C → D

②

```
import { readFile } from 'fs';
console.log('Welcome to line counter!');

A readFile('input.txt', (err, data) => {
  B const lines = data.split('\n');
    console.log(`${lines.length} lines.`);

  });

C console.log("Also print some numbers!");
  for (let i = 0; i < 10; i++) {
    console.log(i);
  }
```

non-blocking

sync: A → C → B → D

③

```
import { readFile } from 'fs/promises';
console.log('Welcome to line counter!');

A const data = await readFile('input.txt');
B const lines = data.split('\n');
  console.log(`${lines.length} lines.`);

C console.log("Also print some numbers!");
  for (let i = 0; i < 10; i++) {
    console.log(i);
  }
```

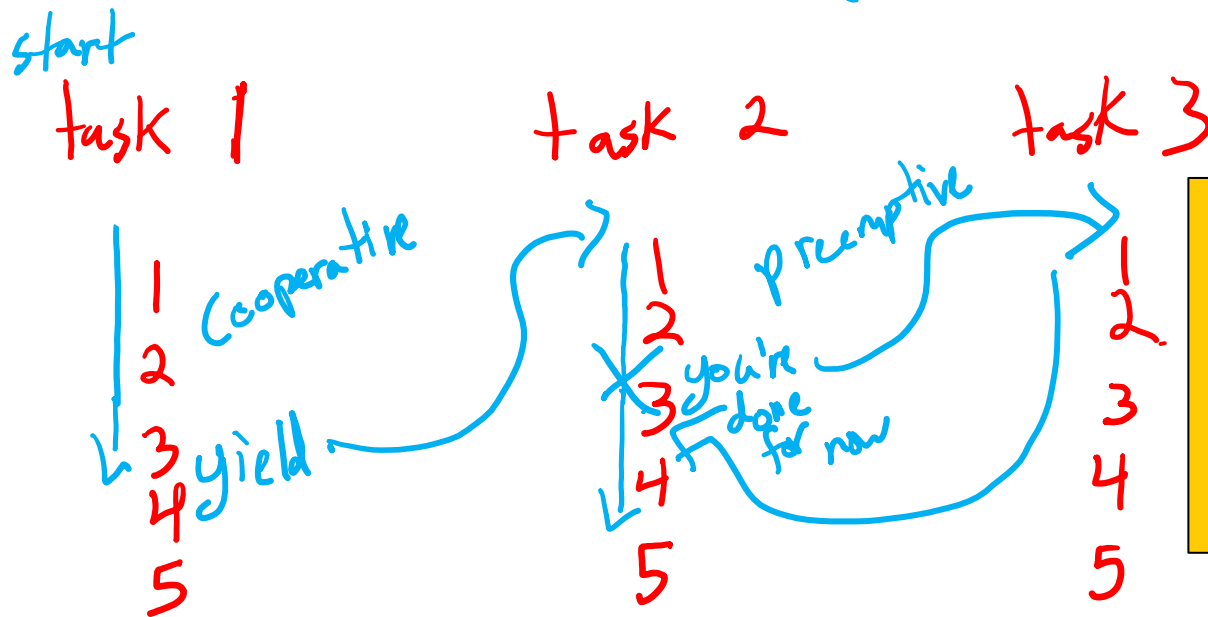
blocks rest of this code, not main thread

sync: A → B → C → D

Cooperative vs. Preemptive

Multitasking Scheduling on thread

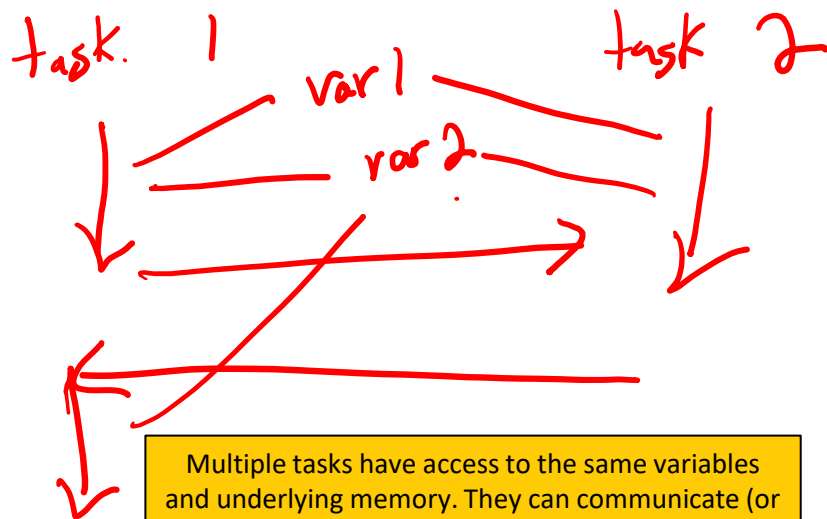
In **cooperative multitasking**, the scheduler can only switch tasks if the individual task yields to let others have a turn.



In **preemptive multitasking**, the scheduler can interrupt a task to switch to others. (There may be ways to protect certain critical operations from being interrupted.)

Shared Memory vs. Message Passing

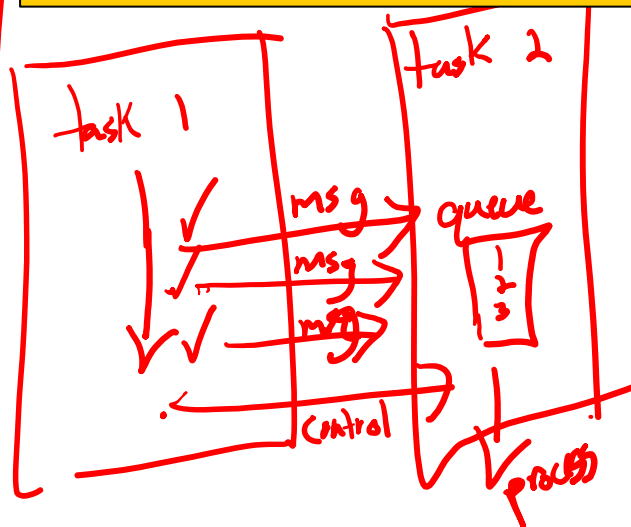
shared memory



Multiple tasks have access to the same variables and underlying memory. They can communicate (or potentially interfere!) through these.

message passing

Communication is only allowed by copying data to be passed through designated message channels. Unintended interference is much less likely, but there is a performance overhead.



Break Time
`bit.ly/4csRS6A`

